

Generative Retrieval and Alignment Model: A New Paradigm for E-commerce Retrieval

Ming Pang^{*†}
Chunyuan Yuan^{*}
pangming8@jd.com
chunyuan93@outlook.com
JD.COM
Beijing, China

Xue Jiang
Changping Peng
jiangxue@jd.com
pengchangping@jd.com
JD.COM
Beijing, China

Xiaoyu He
Zheng Fang
hexiaoyu5@jd.com
fangzheng21@jd.com
JD.COM
Beijing, China

Zhangang Lin
Zheng Luo
linzhangang@jd.com
lawching@jd.com
JD.COM
Beijing, China

Donghao Xie
Fanyi Qu
xiedonghao1@jd.com
qufanyi1@jd.com
JD.COM
Beijing, China

Jingping Shao
shaojingping@jd.com
JD.COM
Beijing, China

Abstract

The retrieval module is a crucial component of search systems. Traditional sparse and dense retrieval methods struggle to leverage general world knowledge and often fail to capture the nuanced features of queries and products. With the advent of large language models (LLMs), industrial search systems have started to employ LLMs to generate identifiers for product retrieval. Commonly used identifiers include (1) static/semantic IDs and (2) product term sets. The first approach requires creating a product ID system from scratch, missing out on the world knowledge embedded within LLMs. While the second approach leverages this general knowledge, the significant difference in word distribution between queries and products means that product-based identifiers often do not align well with user search queries, leading to missed product recalls. Furthermore, when queries contain numerous attributes, these algorithms generate a large number of identifiers, making it difficult to assess their quality, which results in low overall recall efficiency.

To address these challenges, this paper introduces a novel e-commerce retrieval paradigm: the Generative Retrieval and Alignment Model (GRAM). GRAM employs joint training on text information from both queries and products to generate shared text identifier codes, effectively bridging the gap between queries and products. This approach not only enhances the connection between queries and products but also improves inference efficiency. The model uses a co-alignment strategy to generate codes optimized for

maximizing retrieval efficiency. Additionally, it introduces a query-product scoring mechanism to compare product values across different codes, further boosting retrieval efficiency. By integrating these scores, GRAM can replace the traditional recall and initial ranking stages, achieving an integrated retrieval and pre-ranking process. Extensive offline and online A/B testing was conducted. The results demonstrate that GRAM significantly outperforms traditional sparse and dense retrieval algorithms and the latest generative retrieval models, confirming its effectiveness and practicality.

CCS Concepts

• **Information systems** → **Language models**; **Sponsored search advertising**; *Query intent*; **Novelty in information retrieval**; **Online shopping**; • **Computing methodologies** → **Natural language processing**.

Keywords

Large Language Model, Generative Model, E-Commerce Search Information Retrieval

ACM Reference Format:

Ming Pang, Chunyuan Yuan, Xiaoyu He, Zheng Fang, Donghao Xie, Fanyi Qu, Xue Jiang, Changping Peng, Zhangang Lin, Zheng Luo, and Jingping Shao. 2025. Generative Retrieval and Alignment Model: A New Paradigm for E-commerce Retrieval. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3701716.3715228>

1 Introduction

Online shopping has greatly changed our lives and has become an indispensable part of daily life. In the past few years, more and more e-commerce platforms such as Amazon, JD.com, and Taobao have provided consumers with hundreds of millions of colorful products. How to facilitate consumers to quickly and accurately retrieve the products they need from these massive products has become an extremely important research topic.

^{*}Both authors contributed equally to this research.

[†]is corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW Companion '25, April 28-May 2, 2025, Sydney, NSW, Australia

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1331-6/25/04

<https://doi.org/10.1145/3701716.3715228>

Traditional e-commerce retrieval models [4, 19] use sparse retrieval methods such as inverted indexes [4] to retrieve products. With the development of representation learning [1] and deep pre-trained language models [8, 12, 20], dense retrieval has been widely used in online e-commerce retrieval systems. Dense retrieval [7, 27, 28] maps query and product texts to the same vector space for matching. Compared to sparse retrieval, dense retrieval offers improved semantic matching and significantly increases efficiency. However, it has several limitations. First, there is a lack of deep interaction between queries and products, making it difficult to represent fine-grained information, which limits retrieval effectiveness. Second, in industrial scenarios with large product scales, storing representations demands substantial memory. Lastly, model training is constrained by the construction of positive and negative samples, preventing the use of general world knowledge.

With the advancement of large language models, there is growing interest in industrial search systems to explore these models for enhancing the efficiency of their retrieval modules. However, llm-based generative retrieval has not yet been widely adopted in the industry. A central challenge is the selection of identifier codes which can be categorized into two main types:

- **ID-Based identifier codes** [16, 26, 30, 31]: This category includes both static and semantic IDs. Static IDs [30] require the model to directly generate document or product IDs, which are then used to recall the corresponding items. Semantic IDs [16, 26, 31] involve semantically representing the text or product to be recalled, assigning a unique ID to each recalled item through multi-level semantic clustering. While this approach effectively maps a product or document to a unique identifier, it poses significant challenges in terms of generation accuracy and recall efficiency. For instance, recalling k items necessitates the use of beam search to return k results, resulting in a time complexity that can be approximated as $O(k^2)$. Additionally, this approach necessitates learning a new ID system from scratch, which limits the use of the extensive general knowledge embedded in LLMs.
- **String-based identifier codes** [2, 10, 11]: This approach includes sub-strings, which extract n -grams from text as representations, and term sets, which utilize keyword collections for representation. Although these algorithms capitalize on the general knowledge of LLMs, they construct query and product identifiers independently. The significant differences in word distribution between the two text types can severely impact overall recall efficiency. Moreover, when the number of attributes in the query is large, these algorithms can generate excessive identifiers. The quality of these identifiers is often difficult to assess, further diminishing the recall efficiency of the online system.

To address the above challenges, this paper proposes a new e-commerce retrieval paradigm: the Generative Retrieval and Alignment Model (GRAM). We construct product codes based on NER attributes and use a large model to generate query and product identifier codes. These attributes, organized in natural language, fully leverage the general world knowledge embedded in LLM pre-training. By employing joint training on text information from both queries and products, we generate shared text codes, thereby

resolving issues of inconsistent codes caused by differences in word distribution between query and product texts, and enhancing the efficiency of code generation and retrieval. We further assess code quality by examining the variation in recall efficiency of products retrieved via these codes, using a preference alignment algorithm to increase the likelihood of generating high-quality codes, thus boosting retrieval efficiency.

The contributions of this paper are as follows:

- We introduce an innovative and practical approach that utilizes the inherent world knowledge of LLMs to generate shared codes for queries and products, effectively overcoming the inconsistency issues due to differing word distributions in the two domains.
- We develop the GRAM model, which effectively differentiates the quality of codes based on the variation in product recall efficiency during code retrieval, thereby enhancing the model's ability to generate high-quality codes and improve retrieval efficiency. Additionally, by introducing a query-product scoring mechanism, GRAM can simultaneously replace traditional recall and initial ranking stages, achieving an integrated retrieval and pre-ranking process and further boosting retrieval efficiency.
- The effectiveness of GRAM has been validated through extensive offline experiments on a large-scale real-world dataset and online A/B testing. GRAM significantly outperforms traditional sparse and dense retrieval algorithms, as well as the latest generative retrieval models in the industry, confirming its effectiveness and practicality. It has been deployed on the JD e-commerce platform, providing hundreds of millions of product retrieval services every day, showcasing its high commercial value and serving as a practical and robust large-scale product retrieval solution.

2 Related Work

Research on document retrieval can be broadly categorized into three distinct types: sparse retrieval, dense retrieval, and generative retrieval. A brief overview of each category is provided below.

2.1 Sparse Retrieval

Sparse retrieval techniques are fundamental to traditional information retrieval, utilizing inverted indexing to map unique terms to documents efficiently. This method allows for rapid access to relevant information in large collections. A key metric in this domain is the Term Frequency-Inverse Document Frequency (TF-IDF) [17], which assesses the significance of terms across documents and is widely used in retrieval systems. Early research primarily focused on inverted indexing with term-matching metrics such as TF-IDF. BM25 [18] enhanced relevance scoring by refining term weights based on the TF-IDF feature. Recent studies [3, 32] have integrated word embeddings into inverted indexing to address the problem of term mismatch.

Despite the effectiveness of sparse retrieval methods in delivering fast results, they still struggle with intricate queries involving synonyms, specialized terms, or contextual nuances, underscoring the need for ongoing advancements to meet users' diverse information needs better.

2.2 Dense Retrieval

The fundamental principle of dense retrieval is to transform documents and queries into vector representations. The introduction of pre-trained language models, particularly BERT [8], has revolutionized information retrieval, paving the way for dense retrieval methods such as Dense Passage Retrieval (DPR) [7], ColBERT [9], and GTR [13]. Techniques like SimCSE [5] leverage contrastive learning with models such as BERT and Roberta to optimize embeddings. Additionally, dense retrieval methods often employ Approximate Nearest Neighbor (ANN) search [6, 25], Maximum Inner Product Search (MIPS) algorithms [21], and SimLM [24] to ensure efficient retrieval in sub-linear time.

Unlike traditional sparse retrieval, these methods utilize transformer encoders to create dense vector representations for queries and documents, enhancing semantic understanding and retrieval accuracy. This combination of semantic depth and computational efficiency positions dense retrieval as a leading approach in modern information retrieval.

2.3 Generative Retrieval

Generative retrieval has emerged as a promising paradigm in information retrieval, leveraging the advancements in pre-trained models to generate document identifiers (DocIDs) from user queries directly, thus eliminating the need for traditional index-based methods. Early works [23] in this area introduced transformer autoregressive models that preprocess documents into atomic or hierarchical identifiers using hierarchical k-means clustering. Other approaches, like SEAL [2], utilized n-grams as identifiers. Ultron [33] combines keyword-based and semantic-based identifiers within a three-stage training process. TIGER [16] utilizes semantic IDs for product indexing and generates the semantic ID of the next item. Additionally, SE-DSI [22] proposed using summarization texts as document identifiers and RIPOR [30] uses an encoder-decoder model as the backbone where a dense encoder encodes document content and a decoder uses a start token for decoding. GLEN [10] designs dynamic lexical DocIDs and is trained through a two-phase index learning strategy.

These generative retrieval models offer greater flexibility and semantic understanding, enabling end-to-end optimization and reducing dependence on external indexing. However, they either require building product ID systems from scratch, failing to fully utilize the world knowledge embedded in LLMs, or face difficulties in aligning product-based identifiers with user search queries due to significant differences in word distribution. Additionally, when queries have many attributes, these algorithms produce a large number of identifiers, making it challenging to assess their quality, which leads to low overall recall efficiency. Moreover, the generation complexity of the ID-based methods is linear to the beam size, which makes them impractical for industrial applications.

3 Model

In this section, we begin by formally defining the generative retrieval task. Following that, we provide a detailed description of the various modules within GRAM and examine the model's impact during both the training and inference phases.

3.1 Problem Statement

Suppose the query inputted by users on the E-commerce applications, has $q = [q_1, q_2, \dots, q_{L_q}]$ characters. After browsing the search result list, the user clicks product with title $t = [t_1, t_2, \dots, t_{L_t}]$.

Our target is to train a generative retrieval model $f(c|x, \pi_\theta), x \in \{q_i, t_j\}$ to generate product code for any query q_i or product title t_j and ensure that the input query q can retrieve the relevant product t by the code c . Furthermore, considering the retrieval efficiency, we should optimize the generation process to ensure the code c can retrieve as many relevant products for query q as possible. For a clear definition, bold lowercase letters represent vectors throughout the rest of this paper.

3.2 Overview

Figure 1 illustrates the components of the GRAM, which comprises three primary modules: (1) the query-code generator, (2) the product-code generator, and (3) the code co-alignment module. Specifically, the query-code generator takes a query as input and generates a series of predefined text identifiers, which serve as codes that bridge the connection between the query and products. Similarly, the product-code generator uses the product title as input to generate codes that denote the product itself. The code co-alignment module is designed to ensure that the codes generated by both the query and its associated products align effectively, thereby maximizing the overall retrieval efficiency of the search system.

3.3 Code Definition and Construction

In this subsection, we will formally define the composition of the query and product code and how to construct the initial codes to drive the GRAM training process.

3.3.1 Code Definition. Previous research has examined various coding approaches based on identifiers (IDs) [16, 26, 30, 31] and strings [2, 10, 11]. ID-based methods require training product ID systems from scratch, which limits their ability to take advantage of the extensive world knowledge embedded in large language models (LLMs). This limitation significantly undermines the potential effectiveness of LLMs. Therefore, we propose using strings as a coding mechanism to represent products and articulate the intent behind user queries.

In contrast to previous studies [2, 10, 11], we do not employ substrings or n-grams derived from queries or product titles as codes. Instead, we focus on the structural attributes of both queries and products to accurately convey user intent and product characteristics, as both are fundamentally expressed through these structural attributes. Specifically, we have identified 16 commonly used structured attributes as the foundational elements of our coding system: brand names, product categories, series, models, functional attributes, material attributes, style attributes, color attributes, sales specifications, technical specifications, applicable time, product audiences, applicable scenarios, additional modifiers, and marketing terminology. Using these structural attributes enables us to effectively describe nearly any product within the e-commerce system.

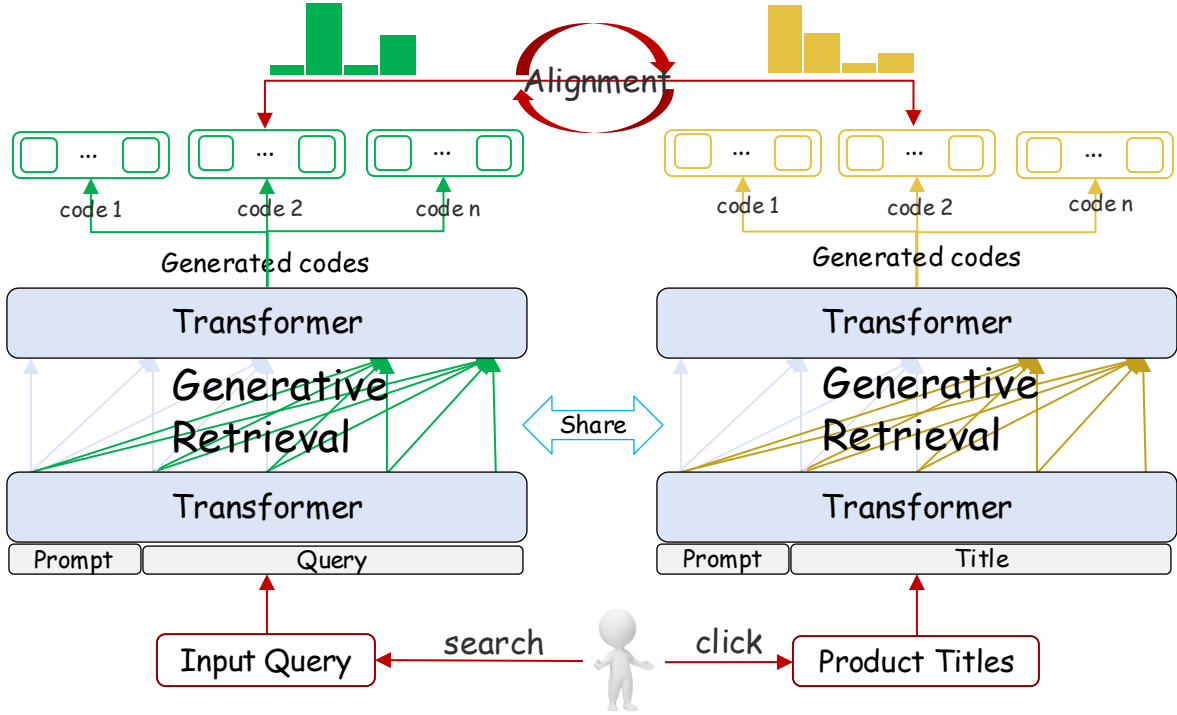


Figure 1: The architecture of the generative retrieval and alignment model.

The coding system is constructed from one or more structured attributes, which are delineated by separators. We define the granularity of the code as follows:

- Coarse-grained code: consists of 1-2 structured attributes connected by delimiters;
- Medium-grained code: consists of 3 structured attributes connected by delimiters;
- Fine-grained code: consists of more than 3 structured attributes connected by delimiters.

3.3.2 Initial Code Construction. First, we use experts to annotate the query with the predefined key attributes above. The results of the two-person labeling are used as reliable labeling samples. The scale of the labeled data is about 3 million. Secondly, we used these labeled data to train a BERT-based NER model. Subsequently, we use the NER discriminant model to extract the key attributes defined above from the query to form a set of key attributes Q . Then, we mine the high-frequency query-product pair click data based on the query click log. At the same time, we use the NER discriminant model to extract the corresponding attributes from the titles of these products. It is necessary to ensure that the extracted attributes are in the set Q .

The initial code of the query consists of two parts: (1) the key attribute combination extracted by the query through the NER model; and (2) the key attribute combination extracted by the product associated with the query through the click relationship. The initial code of the product consists of two parts: (1) the key attribute combination extracted by the product through NER; and (2) the key

attribute combination extracted by the query that is reversely associated with the product through the click relationship. The initial query-code pair and product-code pair data constructed in this way serve as the GRAM's initial training data, driving the entire model's interactive training. The initial training dataset contains about 6 million unique queries and 8 million unique products extracted from the search log of e-commerce applications.

3.4 Supervised Fine-Tuning

In this section, we will outline the process of training the query-code generator and the product-code generator separately, as well as how to utilize these generators for product retrieval. In addition, we will discuss the iterative process of integrating the two generators to enhance the generation of code for both components.

The primary goal of supervised fine-tuning (SFT) training at this stage is to improve the retrieval rate of products retrieved through query-based code generation.

3.4.1 Query-code generator SFT. The query-code generator is trained by the query-code pair data constructed above, with the query text as input and the code text as the target. The open-source LLM is used as the base model for further training. To further improve the generation performance of the model for e-commerce text, we introduce task-specific supervised fine-tuning (SFT). The process of generating text with a conditional language model can be conceptualized as a constrained auto-regressive sampling strategy. Given a query q and its corresponding gold standard code c , we design a special prompt text $Pmpt_q$ to instruct training, the training objective is to maximize the conditional probability $Pr(c|Pmpt_q, q)$.

Specifically, the training objective for the query-code generation model involves minimizing the negative log-likelihood, which can be formulated as follows:

$$\mathcal{L}_{\text{SFT}}^q = -\mathbb{E}_{(q,c) \sim \mathcal{D}_{\text{SFT}}^q} \sum_{i=1}^{L_c} \log \Pr(c_i | \text{Pmpt}_q, q, c_{<i}), \quad (1)$$

where c_i is the i -th token of the code c , $\mathcal{D}_{\text{SFT}}^q$ is the collected query-code training data, L_c is the length of the query code.

3.4.2 Product-code generator SFT. The product-code generator is trained using the product-code pair data constructed above, with the product title text as input and the code text as the target. The process of generating text with a conditional language model can be conceptualized as a constrained auto-regressive sampling strategy. Given a product t and its corresponding gold standard code c , we design a special product prompt Pmpt_t to instruct GRAM training, the objective is to maximize the conditional probability $\Pr(c | \text{Pmpt}_t, t)$. Specifically, the training objective for the product-code generator model involves minimizing the negative log-likelihood, which can be formulated as follows:

$$\mathcal{L}_{\text{SFT}}^t = -\mathbb{E}_{(t,c) \sim \mathcal{D}_{\text{SFT}}^t} \sum_{i=1}^{L_c} \log \Pr(c_i | \text{Pmpt}_t, t, c_{<i}), \quad (2)$$

where c_i is the i -th token of the code c , $\mathcal{D}_{\text{SFT}}^t$ is the collected product-code training data, L_c is the length of the product code.

3.4.3 Generator Co-training. Through the training process outlined above, the GRAM acquires fundamental capabilities for generating query-to-code and product-to-code mappings. We randomly selected a batch of queries and products to generate their corresponding codes. Due to the limited diversity of queries and the small number of products represented in the training data, the model predominantly generated fine-grained codes for both queries and products. Although these fine-grained codes improve the relevance of retrieved products, they can impede retrieval efficiency, as many mid- and long-tail queries and products may not be effectively captured by the generated codes.

To address these issues, we utilized the trained product-code generator to create code data for the active products within the e-commerce system. Simultaneously, leveraging the co-click relationship, we fed the queries associated with these products into the query-code generator, generating 10 new codes for each query. After deduplicating the newly generated codes, they were evaluated using a trained query-code and product-code relevance model. The codes that passed relevance filtering were incorporated into the existing set of codes as augmented training data.

Unlike the separate training conducted in the previous stages, this phase needs to strengthen the connection between the two models, thus training the two generators simultaneously. The objective function for this training is formulated as follows:

$$\mathcal{L}_{\text{SFT}} = \mathcal{L}_{\text{SFT}}^q + \lambda * \mathcal{L}_{\text{SFT}}^t, \quad (3)$$

where λ is a hyper-parameter to adjust the weight of the loss function for the title-code generator.

3.5 Query-Code-Product Co-Alignment

In this section, we will describe the process of aligning the codes generated by the query code generator with those produced by the product code generator. This alignment is intended to enhance the retrieval rate of products through query-based code generation. Additionally, we will discuss how to achieve preference alignment between queries and products within the same code framework, which will contribute to an increased proportion of recalled products that successfully pass through the query-product relevance filtering module.

The goal of co-alignment is to maximize the proportion of retrieved products that pass the query-product relevance filtering module while minimizing any potential decrease in the recall rate of products retrieved through query-based code generation.

3.5.1 Codes Co-Alignment. To achieve the overall goal of this phase, the first step is to align the codes generated by the query-code generator with those produced by the product-code generator. Firstly, we take users' click data from the search log to build a query-product pair dataset. Subsequently, we take the query-code and product-code corresponding to each query-product pair from the initial query-code and product-code training dataset. Finally, we take the intersection of the two datasets as the positive example set, and the difference between the two as the negative example set to build a partial-order dataset $\mathcal{D}_{CA}$.

There are two main considerations why we directly use the query-code pair and product-code pair from the initial training data. The first reason is to align with the initial version of the model and directly correct the results of the model SFT based on the original data; the other is that there are more low-quality codes (especially on the query side) in the initial data, and the probability of selecting pseudo-negative examples during random selection is lower overall.

A single training sample consists of a query q , product t , positive code c_w , and negative code c_l . Direct preference optimization algorithm [15] is used for code alignment. The GRAM, which is trained in the supervised fine-tuning process, is used as the reference model $\pi_{\text{SFT}}(\cdot)$ to calculate the probability values.

$$\begin{aligned} \pi_{\text{SFT}}(c_w | q, t) &= (\pi_{\text{SFT}}(c_w | q) + \pi_{\text{SFT}}(c_w | t)) / 2, \\ \pi_{\text{SFT}}(c_l | q, t) &= (\pi_{\text{SFT}}(c_l | q) + \pi_{\text{SFT}}(c_l | t)) / 2. \end{aligned} \quad (4)$$

Similarly, $\pi_{\theta}(\cdot)$ is applied to calculate the generation probability of positive code and negative code under query, and the generation probability of positive code and negative code under products. Specifically, the process can be formulated as follows:

$$\begin{aligned} \pi_{\theta}(c_w | q, t) &= (\pi_{\theta}(c_w | q) + \pi_{\theta}(c_w | t)) / 2, \\ \pi_{\theta}(c_l | q, t) &= (\pi_{\theta}(c_l | q) + \pi_{\theta}(c_l | t)) / 2. \end{aligned} \quad (5)$$

After obtaining the probabilities, the overall training objective can be formulated as follows:

$$\begin{aligned} \mathcal{L}_{CA}(\pi_{\theta}) &= -\mathbb{E}_{(q,t,c_w,c_l) \sim \mathcal{D}_{CA}} \left[\log \sigma \left(\beta_w \log \frac{\pi_{\theta}(c_w | q, t)}{\pi_{\text{SFT}}(c_w | q, t)} \right. \right. \\ &\quad \left. \left. - \beta_l \log \frac{\pi_{\theta}(c_l | q, t)}{\pi_{\text{SFT}}(c_l | q, t)} \right) \right]. \end{aligned} \quad (6)$$

In the experiment, we found that the probability of fine-grained codes being selected as negative examples is higher than that of coarse-grained codes, and the probability of coarse-grained codes being selected as positive examples is also higher than that of fine-grained codes. This bias may be reasonable for a single piece of data, but it will cause the generated results to be more biased towards coarse-grained codes overall. Therefore, we perform a second data downsampling. For all positive codes matched under the query-product pair, we add a length penalty factor $\alpha = \frac{code_l}{max_{code_l}}$. Finally, the number of repetitions of a positive code in the training data is $n = \sqrt{k} * \alpha$.

3.5.2 Query-product Alignment through Code. The above data construction method and model training method use query-product as anchor points and select positive and negative example codes for preference alignment training. In this way, the model can generate a higher probability for codes with display/click behaviors than the codes without display/click behaviors, which is more about learning the partial order relations of code. However, the partial orders between the query and product under the code are still unclear, so further model alignment is needed.

Based on the model trained in the previous step, recall the product set for any query-code pair. Use the online correlation results to filter out positive and negative products from the recalled product set. Based on the query-code results and product-code results, complete the query-product recall through code matching. For such a set of (query, code, product) triples, calculate the query-product score by token based on the Jensen-Shannon divergence.

For the same token of code, the distance between the probability of query and product generating this token, we use the distance as the basis for scoring. The process can be formulated as follows:

$$S_{rele}(q, t) = \sum_{i=1}^n w_i * \left(P_i^q * \ln \frac{2P_i^q}{P_i^q + P_i^t} + P_i^t * \ln \frac{2P_i^t}{P_i^q + P_i^t} \right), \quad (7)$$

where P_i^q is the probability of generating the i -th token in the query generated code is a scalar between 0 and 1. P_i^t is the probability of generating the i -th token in the product generation code. w_i is code-granular, and each code has a weight. The model parameters are fixed and only the weights are trained.

A product may be retrieved by multiple codes of the same query. Each code will have a relevance score S_{rele} , and the scores of all codes need to be summed up as the relevance of the product.

3.5.3 Overall Alignment Objective. After obtaining the query-product scores through the above process, we construct a partial order relationship of the products retrieved by the query through the code. At the same time, each query uses the top k products with the highest scores as positive samples t_{pos} and the other samples as negative samples t_{neg} . Using these query-product relevance data, we use the pairwise loss as the training objective:

$$\mathcal{L}_{rele} = \max(0, S_{rele}(q, t_{neg}|c) - S_{rele}(q, t_{pos}|c) + \mu), \quad (8)$$

where μ is the margin of the pairwise loss.

4 Experiment

This section will discuss the offline and online experiments in detail. We first introduce the datasets and the evaluation metrics used in

Table 1: Dataset statistics.

Statistics	SFT Dataset	Alignment Dataset
#Query-Code Pairs	184.8M	67.4M
#Product-Code Pairs	459.7M	134.8M
#Uniq. Queries	6.2M	1.5M
#Uniq. Products	8.4M	15.6M
#Uniq. Codes	7.4M	452.7K
Avg. #chars of query	8.0	7.0
Avg. #chars of product	50.3	51.9
Avg. #chars of code	9.9	7.9

this paper. Then, we analyze the experiment results by several fair comparisons with strong baselines. After that, we deeply investigate the effect of different modules of the GRAM model. Subsequently, we present the online performance of the model on the JD search engine and further analyze the influence of various modules. Finally, we explore the influence of hyper-parameters.

4.1 Dataset

To evaluate the effectiveness and generality of the proposed model, we conducted a series of experiments on two large-scale real-world datasets collected from users' click logs on the JD application. The statistics of the datasets are listed in Table 1. Specifically,

- **SFT Dataset:** Based on online click data, we collected query and product pairs from the past month. Using the results of named entity recognition (NER), we constructed initial query-code and product-code associations. Queries and products that co-occurred were linked to their respective 184.8M query-code pairs and 459.7M product-code pairs. The dataset includes 6.2 million queries, 7.4 million codes, and 8.4 million products.
- **Alignment Dataset:** From the online impression logs, we extracted click data from the past seven days and highly relevant impression data from the first three pages to construct a query-product pair dataset. Using SFT training data, we obtained query-code and product-code information. For each query-product pair, we identified the intersection of their tags as the positive example set and the difference as the negative example set, thereby constructing a partially ordered dataset.

4.2 Baseline Models

We compare GRAM with several strong baseline models, including widely used sparse retrieval methods, dense retrieval methods, and generative retrieval methods. The detailed introductions are listed as follows:

- **BM25** [18]: It enhances relevance scoring by refining term weights based on the TF-IDF feature.
- **DocT5Query** [14]: It utilizes T5 to generate a pseudo query for document to expand document information and then applies BM25 for document retrieval.
- **DPR** [7]: It uses BERT as an encoder to encode queries and documents into semantic space and train models with in-batch negatives.

- **SEAL** [2]: It regards all n-grams contained in documents as their identifiers.
- **LC-Rec** [31]: It utilizes the RQ-VAE [29] to generate semantic IDs for product indexing and proposes a series of semantic alignment tasks to align LLM with semantic IDs.

Table 2: The experimental results compared with sparse retrieval methods, dense retrieval methods, and generative retrieval methods.

Models	Recall@10	Recall@100	Recall@300	ReIR
BM25	3.01%	10.52%	15.23%	35.78%
DocT5Query	3.13%	10.88%	15.88%	35.56%
DPR	3.89%	11.26%	17.92%	30.96%
SEAL	3.25%	11.62%	16.56%	27.03%
LC-Rec	4.35%	7.16%	7.33%	23.94%
GRAM	2.85%	12.54%	21.13%	40.18%

4.3 Evaluation Metrics

Recall@k is a metric that measures the proportion of expected documents retrieved by the search system. For a given cutoff point k, Recall@k is defined as:

$$Recall@k = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{ret_{q,k}}{rel_q}, \quad (9)$$

where $|Q|$ is the number of queries in the set, $ret_{q,k}$ is the number of expected documents retrieved for the q -th query within the top k results, and rel_q is the total number of expected documents for the q -th query. The expected documents for one query are documents that users have clicked.

The relevance ratio (ReIR) is a metric that measures the proportion of relevant documents retrieved by the search system. ReIR differs from recall@k in that the relevance ratio does not rely on post-hoc user clicks, allowing for a more comprehensive evaluation of a method’s effectiveness from a relevance perspective.

4.4 Experiment Settings

For LLM fine-tuning and alignment, we implement the models based on the Pytorch framework and utilize an internal 0.5B LLM to generate query code and product code. We use AdamW optimizer with a learning rate of $5e^{-5}$. The max length of the query is set to 16 and the maximum number of codes is set to 10 and each code can contain up to 6 attributes. During the Supervised Fine-Tuning phase, the weight parameter λ in the loss function is set to 1.

We use the dropout strategy with a dropout rate of 0.05 to overcome the overfitting. The maximum training epoch is set to 3, and the batch size of the training set is set to 128. We select the best parameter configuration based on the performance of the validation set and evaluate the configuration on the test set.

4.5 Offline Evaluation

4.5.1 Offline performance. The experimental results are shown in Table 2. Overall, the experimental results indicate that GRAM

significantly outperforms all baselines on a large-scale real-world dataset. Specifically, we have the following observations:

(1) Compared with the sparse and dense retrieval methods (i.e., BM25, DocT5Query, DPR), it is obvious that GRAM outperforms them by a significant margin on the dataset. Sparse retrieval methods mainly focus on character-granular matching. It performs well for frequently searched queries. However, they still struggle with intricate queries involving synonyms, and specialized terms, underscoring the need for ongoing advancements to meet users’ diverse information needs better. Dense retrieval is limited by the construction of positive and negative samples, and general world knowledge cannot be used.

(2) Compared with generative retrieval methods, GRAM demonstrates superior effectiveness and practicality. SEAL utilizes all n-grams as codes without achieving alignment between queries and SKUs, which results in reduced retrieval efficiency and lower recall rates. The LC-Rec method, on the other hand, employs semantic IDs that necessitate aligning general language with these IDs from scratch, heavily depending on training data and potentially diminishing the relevance ratio. Additionally, the complexity of generating LC-Rec is linear concerning the number of recalled products. In contrast, the GRAM method effectively integrates the capabilities of general language understanding by aligning queries and products through shared codes. It further enhances performance by aligning with relevance metrics specific to the retrieval domain, thereby achieving superior results.

In conclusion, GRAM demonstrates a substantial improvement over all baseline models in terms of Recall@k, and relevance. The results confirm that GRAM can enhance retrieval efficiency and query-product relevance simultaneously.

Table 3: Ablation study of the proposed model GRAM. (1) Co-training operation (CT): This component aligns code generation on both the query and product sides. (2) Co-alignment operation (CA): This component aligns offline generation with online relevance.

Models	Recall@10	Recall@100	Recall@300	ReIR
GRAM	2.85%	12.54%	21.13%	40.18%
w/o. CA	1.80%	11.37%	20.23%	33.51%
w/o. CT&CA	1.57%	7.64%	12.89%	33.36%

4.5.2 Ablation study. To further figure out the relative importance of each module in the proposed model, we perform a series of ablation studies over the different components of GRAM. Two variants of GRAM are listed below:

- **w/o Co-alignment operation:** When removing the Co-alignment process, the model exhibits a consistent decline in recall rate, along with a significant decrease in relevance result. This indicates that the co-alignment operation can significantly improve the model’s capability to capture similarities between queries and products.
- **w/o Co-training & Co-alignment operation:** When we eliminate both Co-training and Co-alignment operation, the performance degrades more than 8% in recall@300 compared

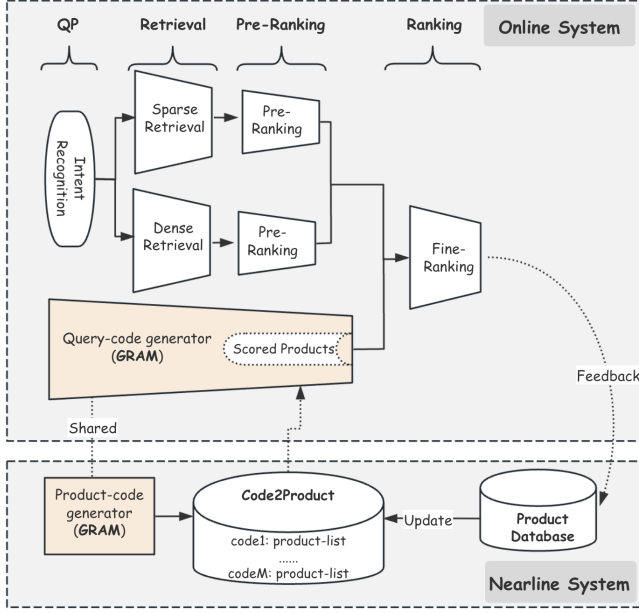


Figure 2: The deployment of GRAM and the role of retrieval plays in the E-commerce system.

with the complete GRAM. The results indicate that improving the interaction between query-code and product-code can significantly enhance retrieval accuracy.

4.6 Online Evaluation

4.6.1 Online Deployment. In online platforms for search advertising, the process is primarily divided into two components: retrieval and ranking. The online system already includes retrieval branches such as sparse retrieval methods like BM25 and DocT5query, as well as dense retrieval methods like DPR.

In this study, we introduce a generative retrieval method in the retrieval phase. To mitigate latency issues, we utilize a real-time cache to reduce the number of inferences, and we set the beam search size to 10, ultimately returning 300 advertisements. Compared to the semantic ID generation approach, this method generates results at a scale of an order of magnitude smaller, making it more suitable for large-scale industrial applications.

Figure 2 illustrates the role of GRAM within the system. GRAM effectively merges the stages of intent recognition, retrieval, and pre-ranking, thereby reducing the information loss typically associated with the layered funnel approach. The same GRAM model operates in both online and nearline systems. Initially, GRAM generates relevant codes based on queries. These codes are then used to ultimately produce items with associated scores. In the nearline component, it generates codes for new products in response to changes in inventory and updates these codes in the product index.

4.6.2 Online Performance. Before being launched in production, we routinely deployed the GRAM online on the JD search engine and made it randomly serve 5% traffic as the test group. For a fair comparison, we also build a base group that using the previous model serves 5% traffic. During the A/B testing period, we monitor

the performance of GRAM and compare it with the online model. This period lasts for at least one week.

For online evaluation, we use some business metrics: Ad. imp.(total count of ad impressions), CPC (cost per click), CTR (click-through rate), and Ad. revenue(total ad revenue). The specific computation of CPC and CTR are defined as follows:

$$CPC = \frac{Ad.revenue}{\#Clicks}, \quad (10)$$

$$CTR = \frac{\#Clicks}{\#Impressions},$$

where #Clicks represents the total number of user clicks on an advertisement, while #Impressions denote the total number of times the advertisement is displayed to users.

Table 4: Online improvements of the GRAM. Improvements are statistically significant with $p < 0.05$ on paired t-test. All performances of GRAM and its variants GRAM without Co-training (CT) and without Co-alignment (CA) are compared with the online model.

Models	Ad. imp.	CTR	CPC	Ad. revenue
Online	-	-	-	-
GRAM w/o. CT&CA	+0.37%	+0.21%	+0.79%	+1.37%
GRAM w/o. CA	+0.56%	+0.92%	+0.68%	+2.16%
GRAM	+0.74%	+1.27%	+0.45%	+2.46%

Compared to the existing retrieval branches in the online system, such as sparse retrieval methods like BM25 and DocT5query, and dense retrieval methods like DPR, GRAM has delivered significant improvements in advertising revenue. It has achieved substantial gains in click-through rate (CTR), cost per click (CPC), and overall ad revenue, validating the effectiveness of GRAM. The experimental results comparing GRAM and its variant models align with the ablation study, demonstrating the effectiveness of dual alignment: the alignment of codes between queries and products, and the alignment of code generation with online relevance.

5 Conclusion and Future Work

In this paper, we have introduced the Generative Retrieval and Alignment Model (GRAM), a novel approach to e-commerce retrieval that leverages the capabilities of large language models (LLMs) to generate shared codes for both queries and products. By effectively addressing the challenges of inconsistent code generation due to differing word distributions, GRAM enhances retrieval efficiency and accuracy. Our approach integrates retrieval and pre-ranking processes, significantly outperforming traditional sparse and dense retrieval methods, as well as existing generative retrieval models. The deployment of GRAM on the JD e-commerce platform, where it handles millions of product retrievals daily, underscores its commercial viability and scalability.

For future work, we aim to further unleash the potential of large language models by incorporating multi-modal and personalized information into the GRAM framework. Building on the integration of retrieval and pre-ranking, a promising goal is to develop an end-to-end recommendation system that encompasses the entire decision-making chain.

References

- [1] Yoshua Bengio, Aaron Courville, and Pascal Vincent. 2013. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence* 35, 8 (2013), 1798–1828.
- [2] Michele Bevilacqua, Giuseppe Ottaviano, Patrick Lewis, Scott Yih, Sebastian Riedel, and Fabio Petroni. 2022. Autoregressive search engines: Generating substrings as document identifiers. *Advances in Neural Information Processing Systems* 35 (2022), 31668–31683.
- [3] Zhuyun Dai and Jamie Callan. 2020. Context-aware document term weighting for ad-hoc search. In *Proceedings of The Web Conference 2020*. 1897–1907.
- [4] Jeffrey Dean. 2009. Challenges in building large-scale information retrieval systems. In *Keynote of the 2nd ACM International Conference on Web Search and Data Mining (WSDM)*, Vol. 10.
- [5] T Gao, X Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *EMNLP 2021-2021 Conference on Empirical Methods in Natural Language Processing, Proceedings*.
- [6] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems* 32 (2019).
- [7] Vladimir Karpukhin, Barlas Ögüz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen Tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020*. Association for Computational Linguistics (ACL), 6769–6781.
- [8] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.
- [9] Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 39–48.
- [10] Sunkyoung Lee, Minjin Choi, and Jongwuk Lee. 2023. GLEN: Generative Retrieval via Lexical Index Learning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7693–7704.
- [11] Yongqi Li, Zhen Zhang, Wenjie Wang, Liqiang Nie, Wenjie Li, and Tat-Seng Chua. 2024. Distillation Enhanced Generative Retrieval. *arXiv preprint arXiv:2402.10769* (2024).
- [12] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [13] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, et al. 2022. Large Dual Encoders Are Generalizable Retrievers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 9844–9855.
- [14] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. 2019. From doc2query to docTTTTTquery. (2019).
- [15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024).
- [16] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [17] Juan Ramos et al. 2003. Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning*, Vol. 242. Citeseer, 29–48.
- [18] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [19] Stephen E Robertson and Steve Walker. 1994. Some simple effective approximations to the 2-poisson model for probabilistic weighted retrieval. In *SIGIR'94*. Springer, 232–241.
- [20] Jürgen Schmidhuber. 2015. Deep learning in neural networks: An overview. *Neural networks* 61 (2015), 85–117.
- [21] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). *Advances in neural information processing systems* 27 (2014).
- [22] Yubao Tang, Ruqing Zhang, Jiafeng Guo, Jiangui Chen, Zuwei Zhu, Shuaiqiang Wang, Dawei Yin, and Xueqi Cheng. 2023. Semantic-enhanced differentiable search index inspired by learning strategies. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4904–4913.
- [23] Yi Tay, Vinh Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, et al. 2022. Transformer memory as a differentiable search index. *Advances in Neural Information Processing Systems* 35 (2022), 21831–21843.
- [24] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2023. SimLM: Pre-training with Representation Bottleneck for Dense Passage Retrieval. In *The 61st Annual Meeting Of The Association For Computational Linguistics*.
- [25] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N Bennett, Junaid Ahmed, and Arnold Overwijk. [n. d.]. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations*.
- [26] Tianchi Yang, Minghui Song, Zihan Zhang, Haizhen Huang, Weiwei Deng, Feng Sun, and Qi Zhang. 2023. Auto search indexer for end-to-end document retrieval. *arXiv preprint arXiv:2310.12455* (2023).
- [27] Chunyuan Yuan, Ming Pang, Zheng Fang, Xue Jiang, Changping Peng, and Zhangang Lin. 2024. A Semi-supervised Multi-channel Graph Convolutional Network for Query Classification in E-commerce. In *Companion Proceedings of the ACM on Web Conference 2024*. 56–64.
- [28] Chunyuan Yuan, Yiming Qiu, Mingming Li, Haiqing Hu, Songlin Wang, and Sulong Xu. 2023. A multi-granularity matching attention network for query intent classification in e-commerce retrieval. In *Companion Proceedings of the ACM Web Conference 2023*. 416–420.
- [29] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2021), 495–507.
- [30] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2024. Scalable and effective generative information retrieval. In *Proceedings of the ACM on Web Conference 2024*. 1441–1452.
- [31] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
- [32] Guoqing Zheng and Jamie Callan. 2015. Learning to reweight terms with distributed representations. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 575–584.
- [33] Yujia Zhou, Jing Yao, Zhicheng Dou, Ledell Wu, Peitian Zhang, and Ji-Rong Wen. 2022. Ultron: An ultimate retriever on corpus with a model-based indexer. *arXiv preprint arXiv:2208.09257* (2022).